

[통합본]이미지 데이터 연동 및 3D 시공간 물리 분석 계획서

본 보고서는 비디오 프레임 순서 정렬 과제에서 발생할 수 있는 규칙 기반 하드코딩(Hard-coding)과 도메인 과적합(Overfitting) 문제를 원천 차단하고, **객체의 물리적 기하학(Pure Geometry) 정보**와 **VLM의 의미론적 맥락 이해력**을 결합하여 정렬 정확도를 극대화하는 최종 설계안입니다.

1. 기하학적 시공간 물리 분석 (Geometric Spatial Analysis)

A. 핵심 아이디어: 규칙 배제 및 순수 기하 특징 추출

카메라 기법(줌인/줌아웃) 및 객체의 이동 궤적을 임의의 텍스트 매핑 규칙(if-else)으로 정렬하려는 시도는 평가 데이터셋의 도메인 변화에 취약합니다. 본 설계는 도메인에 종속되지 않는 보편적인 물리 법칙인 **이미지 평면 상의 기하학적 정보**만을 추출합니다.

- **Z축 깊이 대응 특징 (R_{bbox}):** 단안 깊이 모델(Depth Anything v2)의 scale-shift bias에 의한 왜곡을 피하기 위해, 객체 바운딩 박스가 전체 이미지에서 차지하는 면적 비율(R_{bbox} =객체 면적/이미지 면적)을 절대적인 크기 변화 기준으로 삼아 줌인/줌아웃을 안정적으로 검출합니다.

R_{bbox} =객체 면적/이미지 면적

- **X, Y축 평면 궤적 특징 (x_c, y_c):** BBox의 중심점 좌표를 이미지 해상도 대비 0.0~1.0 범위로 정규화하여 좌우/상하 이동 흐름을 정량화합니다.

0.0~1.0

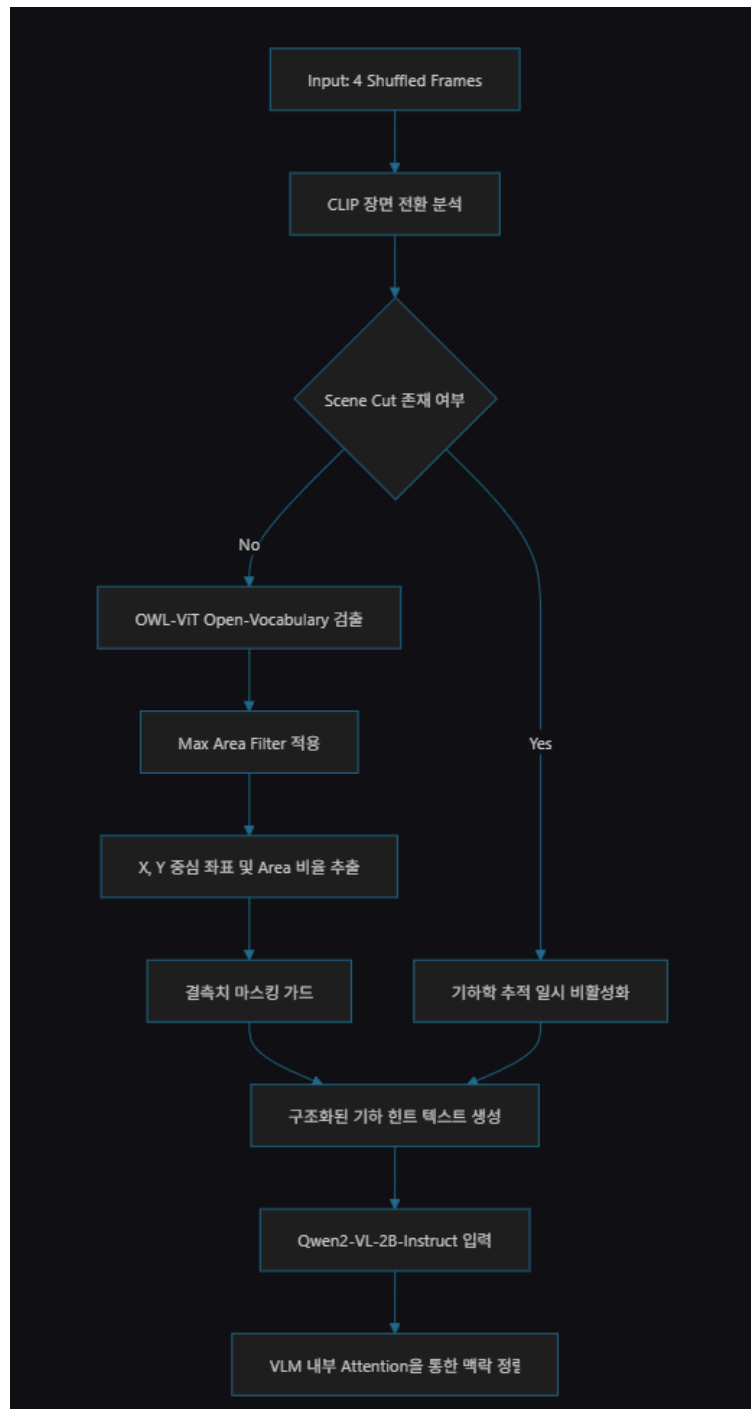
B. 추출 방법론 (Preprocessing Pipeline)

1. **Open-Vocabulary 객체 탐지:** [google/owlvit-base-patch32](https://github.com/google/owlvit-base-patch32)를 탑재하여 문장에 명시된 임의의 사물명(예: `kayak`, `scissors`, `comb` 등)을 검출해 YOLO의 80개 단어 제한 문제를 해결합니다.
2. **대표 객체 선정 (Max Area Filter):** 화면에 동일 클래스의 복수 객체가 검출되어 경로가 꼬이는 혼선을 막기 위해, 화면 점유 면적이 가장 큰 객체를 메인 피사체로 일관되게 선택합니다.

3. **결측치 마스킹 (Missed Detection Masking):** 프레임 누락 시 선형보간법을 사용하면 비선형적 움직임 왜곡이나 외삽 오류를 범하므로, 검출 실패 프레임은 가짜 좌표 대신 명시적 **Skip 플래그**를 부여해 배제합니다.
-

2. 역할 분담 및 하드코딩 방지 아키텍처 (Soft Prompting Bridge)

본 파이프라인은 과적합을 막기 위해 모든 종류의 결정 규칙을 파이프라인 코드에서 배제하고 VLM 내부 가중치에 추론을 위임합니다.



- 임베딩 앵커 코사인 정렬 (Dynamic Weights):** 어휘적 과적합(Lexical Overfitting)을 막기 위해 "가까워진다", "오른쪽" 등의 단어 리스트 하드코딩 매핑을 폐기합니다. 입력된 캡션 벡터와 사전 정의된 두 개의 기준 벡터(깊이를 나타내는 'Depth' 축 vs 평면 이동을 나타내는 'Trajectory' 축) 간의 코사인 유사도를 계산하여 어떤 성격의 기하학 힌트를 신뢰할지 동적 가중치 가이드를 제공합니다.
- VLM으로의 전면 위임 (Soft Prompting):** 추출된 정형 수치 데이터를 텍스트 프롬프트 형태로 감싸 **Qwen2-VL**에 입력합니다. VLM의 대형 트랜스포머 블록이 문장의 세밀한

행동 묘사(예: "걸어오면서 고개를 숙인다")를 우리가 제공한 좌표 트렌드($X, Y, Area$ 변화)와 스스로 대조해 교차 검증하도록 유도합니다.

$X, Y, Area$

3. 파이프라인 구현 코드 (Python)

```
import os
import torch
import numpy as np
from PIL import Image
from transformers import OwlViTProcessor, OwlViTForObjectDetection

# OpenMP duplicate runtime fix for PyTorch on Windows
os.environ['KMP_DUPLICATE_LIB_OK'] = 'True'

class Spatial3DOwlViTTrajectoryExtractor:
    def __init__(self, model_name="google/owlvit-base-patch32", device="cpu"):
        self.device = device
        self.processor = OwlViTProcessor.from_pretrained(model_name)
        self.model = OwlViTForObjectDetection.from_pretrained(model_name).to(self.device)
        self.model.eval()

    def extract_3d_spatial_features(self, image_paths, query_text):
        """
        4장의 이미지와 검색 대상 쿼리 텍스트를 이용하여 X, Y 궤적 및 면적 변화 비율을 추출합니다.
        """
        results_summary = []
        text_queries = [[query_text]]

        for idx, img_path in enumerate(image_paths):
            if not os.path.exists(img_path):
```

```

        results_summary.append({
            "frame": idx + 1,
            "status": "file_not_found",
            "bbox": None,
            "center": None,
            "area_ratio": 0.0
        })
        continue

    img = Image.open(img_path).convert("RGB")
    w, h = img.size
    img_area = w * h

    # 1. OWL-ViT 객체 탐지
    inputs = self.processor(text=text_queries, images=img, return_tensors="pt")
    inputs = {k: v.to(self.device) for k, v in inputs.items()}
    with torch.no_grad():
        outputs = self.model(**inputs)

    target_sizes = torch.tensor([img.size[:-1]], dtype=torch.float32).to(self.device)
    results = self.processor.post_process_object_detection(
        outputs=outputs, target_sizes=target_sizes,
        threshold=0.10
    )[0]

    boxes = results["boxes"].cpu().numpy()

    # 2. 검출 실패 시 결측치 마스킹 처리 (가짜 폴백 좌표
    # 오관 방지)
    if len(boxes) == 0:
        results_summary.append({
            "frame": idx + 1,
            "status": "missed_detection",
            "bbox": None,

```

```

        "center": None,
        "area_ratio": 0.0
    })
    continue

# 3. Max Area 필터를 통해 일관된 대표 피사체 고정
best_idx = 0
max_area = 0
for i, box in enumerate(boxes):
    box_w = box[2] - box[0]
    box_h = box[3] - box[1]
    area = box_w * box_h
    if area > max_area:
        max_area = area
        best_idx = i

best_box = boxes[best_idx]
x1, y1, x2, y2 = map(int, best_box)

# 중심 좌표 정규화 (0.0 ~ 1.0) 및 면적 비율 계산
cx = ((x1 + x2) / 2) / w
cy = ((y1 + y2) / 2) / h
best_area_ratio = max_area / img_area

results_summary.append({
    "frame": idx + 1,
    "status": "success",
    "bbox": (x1, y1, x2, y2),
    "center": (cx, cy),
    "area_ratio": best_area_ratio
})

return results_summary

```

4. VLM 프롬프트 주입 힌트 구조

추출된 정량적 기하 수치는 프롬프트 상에서 다음과 같은 종립적 지표 형태로 VLM에 전달되어 과적합 없는 맥락적 판단을 지원합니다.

[시각 분석 보조 시스템 기하학적 시공간 힌트]

4장의 뒤섞인 이미지와 자연어 캡션을 대조하여 올바른 프레임 순서를 추론하세요.

- 캡션 가중치 배분 (임베딩 앵커 코사인 정렬):

* 깊이 방향 유사도 (Depth Match): 0.82

* 평면 궤적 유사도 (Trajectory Match): 0.15

* (시스템 추천: 깊이 변화 흐름[주인공의 화면 점유 비율 변화]에 더 큰 가중치를 두어 순서를 정렬하세요.)

- 물리 측정 데이터 (OWL-ViT):

* Image 1: 'kayak' center=[X=0.512, Y=0.495], Area=31.4%

* Image 2: 'kayak' center=[X=0.505, Y=0.502], Area=18.2%

* Image 3: 'kayak' center=[X=0.490, Y=0.488], Area=11.5%

* Image 4: 'kayak' - no object detected (skip this cue)

[최종 정렬 가이드]

만약 캡션에 피사체가 가까워지거나(Zoom-in) 멀어지는(Zoom-out) 물리적 묘사가 지배적이라면, 검출된 Area 비율 추세(Image 3 (11.5%) -> Image 2 (18.2%) -> Image 1 (31.4%))를 단서로 삼으십시오. (Image 4는 분석 대상에서 배제)

위의 순수 기하학적 수치와 캡션의 전체 맥락을 대조하여 최종 셔플 인덱스 정답 [n, n, n, n]을 도출하세요.